7.1.1. Binary Search Tree Operations

Code :

```
#include <stdio.h>

#include <stdlib.h>


struct TreeNode {

   int val;

   struct TreeNode* left;

   struct TreeNode* right;

};



struct TreeNode* createNode(int data) {

   struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));

   newNode->val = data;

   newNode->left = newNode->right = NULL;

   return newNode;

}


struct TreeNode* insertNode(struct TreeNode* root, int data) {

   if (root == NULL) return createNode(data);


   if (data < root->val)

      root->left = insertNode(root->left, data);

   else

      root->right = insertNode(root->right, data);


   return root;
```

```
}



void inorder(struct TreeNode* node) {
   if (node == NULL) return;
   inorder(node->left);
   printf("%d ", node->val);
   inorder(node->right);
}

void preorder(struct TreeNode* node) {
   if (node == NULL) return;
   printf("%d ", node->val);
   preorder(node->left);
   preorder(node->right);
}

void postorder(struct TreeNode* node) {
   if (node == NULL) return;
   postorder(node->left);
   postorder(node->right);
   printf("%d ", node->val);
}
 void inorderTraversal(struct TreeNode* root) {
   if (root == NULL) {
      printf("The tree is empty\n");
      return;
```

```
    }
    inorder(root);
    printf("\n");
}

void preorderTraversal(struct TreeNode* root) {
    if (root == NULL) {
        printf("The tree is empty\n");
        return;
    }
    preorder(root);
    printf("\n");
}

void postorderTraversal(struct TreeNode* root) {
    if (root == NULL) {
        printf("The tree is empty\n");
        return;
    }
    postorder(root);
    printf("\n");
}
 struct TreeNode* findMin(struct TreeNode* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}
```

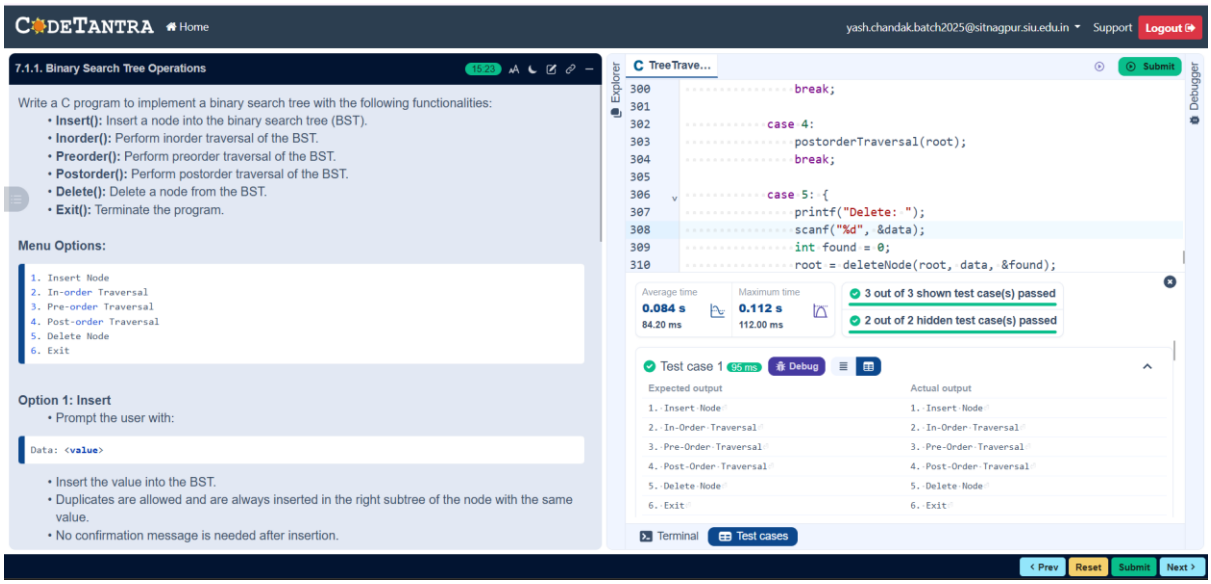
```
struct TreeNode* deleteNode(struct TreeNode* root, int data, int* found) {
  if (root == NULL) return NULL;

  if (data < root->val)
    root->left = deleteNode(root->left, data, found);
  else if (data > root->val)
    root->right = deleteNode(root->right, data, found);
  else {
    *found = 1;

    if (root->left == NULL) {
      struct TreeNode* temp = root->right;
      free(root);
      return temp;
    }
    else if (root->right == NULL) {
      struct TreeNode* temp = root->left;
      free(root);
      return temp;
    }

    struct TreeNode* temp = findMin(root->right);
    root->val = temp->val;
    root->right = deleteNode(root->right, temp->val, found);
  }

  return root;
}
```

```
void freeTree(struct TreeNode* root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

int main() {
    struct TreeNode* root = NULL;
    int choice, data;

    while (1) {
        printf("1. Insert Node\n");
        printf("2. In-Order Traversal\n");
        printf("3. Pre-Order Traversal\n");
        printf("4. Post-Order Traversal\n");
        printf("5. Delete Node\n");
        printf("6. Exit\n");
        printf("Choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Data: ");
                scanf("%d", &data);
                root = insertNode(root, data);
                break;
```

```
case 2:
   inorderTraversal(root);
   break;

case 3:
   preorderTraversal(root);
   break;

case 4:
   postorderTraversal(root);
   break;

case 5: {
   printf("Delete: ");
   scanf("%d", &data);
   int found = 0;
   root = deleteNode(root, data, &found);
   if (!found)
      printf("Value not found\n");
   break;
}

case 6:
   freeTree(root);
   exit(0);

default:
   printf("Invalid choice\n");
```

```
        }
    }
}
```

Programme :


CODETANTRA
Home
yash.chandak.batch2025@sitnagpur.siu.edu.in
Support
Logout
7.1.1. Binary Search Tree Operations
15:23
Write a C program to implement a binary search tree with the following functionalities:
• Insert(): Insert a node into the binary search tree (BST).
• Inorder(): Perform inorder traversal of the BST.
• Preorder(): Perform preorder traversal of the BST.
• Postorder(): Perform postorder traversal of the BST.
• Delete(): Delete a node from the BST.
• Exit(): Terminate the program.
Menu Options:
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Delete Node
6. Exit
Option 1: Insert
• Prompt the user with:
Data: <value>
• Insert the value into the BST.
• Duplicates are allowed and are always inserted in the right subtree of the node with the same value.
• No confirmation message is needed after insertion.
Explorer
TreeTrave...
Submit
Debugger
300 break;
301
302 case 4:
303 postorderTraversal(root);
304 break;
305
306 case 5: {
307 printf("Delete: ");
308 scanf("%d", &data);
309 int found = 0;
310 root = deleteNode(root, data, &found);
Average time
0.084 s
84.20 ms
Maximum time
0.112 s
112.00 ms
3 out of 3 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
95 ms
Debug
Expected output
Actual output
1. Insert Node
2. In-Order Traversal
3. Pre-Order Traversal
4. Post-Order Traversal
5. Delete Node
6. Exit
Terminal
Test cases
Prev
Reset
Submit
Next